Class06: R functions

Hyuntae (Han) An (200231511)

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma seperated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function `add()` to add numbers together.

> **Q1a.** Your first version of the function should add two input numbers together. For example, add(4, 7) should return 11. [1pt]

```
add <- function(x, y=0) {
  x+y
}
```

Let's test our new function

```
add(4,7)
```

```
[1] 11
```

How about:

```
add(10)
```

```
[1] 10
```

```
add <- function(x, y=0, z=0) {
  x+y+z
}
```

```
add(10,10,100)
```

```
[1] 120
```

Q1b. For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)). [1 pt]

```
add <- function(x, y=0) {
  sum(x, y)
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add( c(4,7,10) )
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the … (dots) argument or the base R function sum(). [2 pts]

```
add <- function(...) {
  sum(...)
}
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

A generate DNA function

Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user.

> **Q2a.** Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C". [1 pt]

Useful base R functions that you may (or may not) find useful for this lab include: `sample()`, `paste()`, `paste0()`, `if()`, `for()`, `sapply()`, `nchar()`, and `cat()`.

The `sample()` function looks useful. There are two **required arguemnts** (i.e. inputs I must give that don't have a default value)

```
sample(x=1:10, size=4)
```

```
[1] 4 2 5 1
```

We can use this with "A" "C" "G" and "T" as input x to make random nucleotide sequence:

```
sample(x=c("A","C","G","T"), size=15, replace= TRUE)
```

```
 [1] "A" "G" "A" "C" "A" "G" "T" "A" "C" "A" "C" "G" "G" "C" "T"
```

Cool! We have a working snippet of code that does something useful! This can serve as the "body" of our first `generate_dna()` function.

```
generate_dna <- function(len) {
  sample(x=c("A","C","G","T"), size= len, replace= TRUE)
}
```

```
generate_dna(6)
```

```
[1] "C" "C" "G" "T" "C" "A"
```

> **Q2b.** Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG". [1 pt]

The functions `paste()`, `paste0()`, and `if()` might be useful here…

```
paste(c("A","C","A"), collapse="")
```

```
[1] "ACA"
```

```
flag <- TRUE
if(flag) {
  cat("Look at me! We are in here")
} else {
  cat("Nooooo!")
}
```

```
Look at me! We are in here
```

Let's combine these two things into our nre improved fucntion:

```
generate_dna <- function(len, collapse=FALSE) {
  # Multi-element nucleotide vector
  s <- sample(x=c("A","C","G","T"), size= len, replace= TRUE)

  if(collapse) {
  # Make a single element vector
  s <- paste(s, collapse="")
  #cat("You picked TRUE!!!")
  }
  return(s)
}
```

```
generate_dna(6, collapse=F)
```

```
[1] "A" "T" "C" "A" "A" "G"
```

```
generate_dna(6, collapse=T)
```

```
[1] "AGCTTC"
```

```
generate_dna(6)
```

```
[1] "G" "G" "A" "A" "T" "G"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
len <- 90
paste(">length", len, sep="")
```

```
[1] ">length90"
```

```
generate_dna <- function(len, fasta=FALSE) {

  # Multi-element nucleotide vector
  s <- sample(x=c("A","C","G","T"), size= len, replace= TRUE)

  if(fasta) {
    # Make a single element vector
    s <- paste(s, collapse="")

    ##Add a FASTA >id line
    id <- paste(">legnth", len, sep="")
    s <- paste(id,s,sep="\n")
    cat(s)
  }
  return(s)
}
```

```
x <- generate_dna(10, fasta=T)
```

```
>legnth10
TCTATATACC
```

A new generate protein function

Q3. Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

```
generate_protein <- function(len, fasta=TRUE) {
## FRom page 1 of lab-sheet
aa<- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T",

##Multi-element nucleotide vector
s <- sample(x=aa, size=len, replace=TRUE)

if(fasta) {
  # Make a single element vector
 s <- paste(s,collapse="")

#Add a fasta >id line
id <- paste(">length",len,sep="")
s<- paste(id,s,"\n",sep = "\n")
  cat(s)
  }
  return(s)
}
```

```
generate_protein(6)
```

```
>length6
SQKHCL

[1] ">length6\nSQKHCL\n\n"
```

Generate random protein sequences of length 6 to 13

> **Q4.** Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand.

```
x <- generate_protein(6,fasta=T)
```

```
>length6
MYSQQT
```

```
x <- generate_protein(7,fasta=T)
```

```
>length7
ESAHLFV
```

```
x <- generate_protein(8,fasta=T)
```

```
>length8
SARHVLTP
```

```
x <- generate_protein(9,fasta=T)
```

```
>length9
TKMWFWTKT
```

```
x <- generate_protein(10,fasta=T)
```

```
>length10
IQPDVNQIEY
```

```
x <- generate_protein(11,fasta=T)
```

```
>length11
TDSLACFAWWV
```

```
x <- generate_protein(12,fasta=T)
```

```
>length12
NHPYLKLYIFDS
```

```
x <- generate_protein(13,fasta=T)
```

```
>length13
DFWPMDGFLARCW
```

we can do this with a 'far()' loop

```
for (i in 1:10) {
  cat(i,"\n")

}
```

```
1
2
3
4
5
6
7
8
9
10
```

```
for(len in 6:13) {
  cat(len,"\n")
  cat("length=",len,"\n")
  x<-generate_protein(len,fasta=T)

}
```

```
6
length= 6
>length6
KMKYPC

7
length= 7
>length7
GGHWMVI

8
length= 8
>length8
ATVATIGY

9
length= 9
>length9
```

```
CKWAILRAY

10
length= 10
>length10
KRPVTIYTLV

11
length= 11
>length11
FNIAFSFTLMG

12
length= 12
>length12
PFQGIKIWIRWL

13
length= 13
>length13
EKNQCGEHCWICW
```

```r
sapply(3:13, generate_protein, fasta=T)
```

```
>length3
FKR

>length4
TCHP

>length5
QMIKI

>length6
CKTNAI

>length7
ISSIYYD

>length8
MTPPMEYP
```

```
>length9
VPHDDVNAR

>length10
LTQQPMEIIL

>length11
VCADQNYTVVP

>length12
PPLHQCVHGINS

>length13
RNADFEGIGAMLI
```

```
 [1] ">length3\nFKR\n\n"            ">length4\nTCHP\n\n"
 [3] ">length5\nQMIKI\n\n"          ">length6\nCKTNAI\n\n"
 [5] ">length7\nISSIYYD\n\n"        ">length8\nMTPPMEYP\n\n"
 [7] ">length9\nVPHDDVNAR\n\n"      ">length10\nLTQQPMEIIL\n\n"
 [9] ">length11\nVCADQNYTVVP\n\n"   ">length12\nPPLHQCVHGINS\n\n"
[11] ">length13\nRNADFEGIGAMLI\n\n"
```

BLASTp search against nr — are your peptides "unique in nature"?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/.

Legnth	%cov	%ide	Unique (Y/N)
6	100.00	100	N
7	100.00	100	N
8	100.00	88	Y
9	88.89	100	Y
10	100.00	90	Y
11	100.00	73	Y
12	83.33	92	Y
13	88.89	69	Y

5a. At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)?

My randomly generated peptides start to look "unique in nature" at 8 amino acids. The 6 and 7 aa peptides had hits with 100% identity and 100% coverage, but from 8 aa onward no hit reached both.

> **5b.**Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20 for a peptide of length) and to the size of the known protein universe.

The number of possible peptides grows exponentially and the nr database contains hundreds of millions of proteins, each a few hundred amino acids long, which is far more than the number of possible 6'mers but far fewer than the number of possible longer peptides. So very short random peptides almost always appear somewhere in nr by chance, while longer ones quickly outgrow the known protein universe and are unlikely to have an exact match.

Connecting your findings to immunology (MHC class II and T-cell activation)

> **Q6.** In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Looking through the data presented in Q5, it can be said that the minimum length is around 9 to 13 amino acids. The random peptides of 6 and 8 aa had exact nr matches, while from 8aa onward did not. By 13aa the best hit covered only 69% of the sequence. Presenting very short peptides would be a bad desgin choice, as shown in Q5b, the sequence space for short lengths is small enough that almost any short peptide from a pathogen would also occur somewhere in the host's own proteins.